

TECNOLÓGICO NACIONAL DE MÉXICO



Tecnológico Nacional de México Campus Culiacán

Informe_Proyecto de ventas e inventario en ensamblador.

Equipo 11:

- **José Ángel Grande Espinoza**
- **Jesús Iván Martínez Reyes**
- **Ángel Ernesto Santos Plazola**

Carrera: **Ingeniería En Sistemas Computacionales**

Materia: **Lenguajes de interfaz**

Docente: **Dr. José Luis Medina Jiménez**

Grupo: **8:00 - 9:00 am**

Negocio elegido

Elegimos una tienda de electrónica que vende laptops, pantallas, mouse, audífonos y accesorios de limpieza. El sistema es un programa de consola en MASM x86 de 32 bits con la biblioteca Irvine32. Permite registrar hasta 10 productos, consultarlos y buscarlos por código, recibir mercancía, vender, ver un resumen de la sesión y guardar el catálogo en un archivo para recuperarlo en otra ejecución.

Regla adicional: alerta de existencia mínima

Cuando se aplica: cuando un producto tiene 5 piezas o menos, el programa avisa que hay que resurtirlo. Con 6 piezas o más no hay alerta.

La regla se revisa en tres lugares del programa. Después de cada venta, VenderProducto revisa la existencia que quedó y muestra "ALERTA: quedan 5 piezas o menos, hay que resurtir este producto." o, si no aplica, "Existencia suficiente (mas de 5 piezas), no hay alerta.". Al mostrar un producto en el inventario, en la búsqueda o al registrarlo, MostrarProducto agrega "EXISTENCIA BAJA" al final del renglón. En el resumen, MostrarResumen cuenta cuántos productos están por resurtir.

En los tres casos se usa una comparación cmp eax, 5 seguida de ja, que salta cuando la existencia es mayor que 5. Se usa un salto sin signo porque la existencia nunca es negativa.

Demostración con el archivo inventario2026.txt:

- al vender 1 pieza de ABC-002 (Macbook Pro M5), que tenía 5, quedan 4 y aparece la alerta.
- Al vender 3 piezas de ABC-001 (Laptop Msi), que tenía 20, quedan 17 y el programa indica que no hay alerta.
- Al cargar el archivo, ABC-002 y ABC-004 (con 0 piezas) aparecen marcados con EXISTENCIA BAJA.

Organización de los datos en memoria

Los productos se guardan en un arreglo llamado catalogo de 440 bytes: 10 productos de 44 bytes cada uno, uno detrás de otro. La variable numProd indica cuántos están ocupados. Cada producto tiene sus campos siempre en la misma posición:

Bytes 0 a 7: código, 7 caracteres más el 00h.
Bytes 8 a 33: nombre, hasta 25 caracteres más el 00h.
Bytes 34 y 35: relleno, para que el precio empiece en un múltiplo de 4.
Bytes 36 a 39: precio en centavos (DWORD).
Bytes 40 a 43: existencia (DWORD).

Formato del archivo

El catálogo se guarda en un archivo de texto; para la demostración usamos inventario2026.txt. Cada línea es un producto con 4 campos separados por "|" y termina con salto de línea (0Dh, 0Ah). El precio va en centavos:

ABC-001|Laptop Msi|999999|10
ABC-002|Macbook Pro M5|800000|5
ABC-003|Limpiador de pantallas|1|99
ABC-004|Pantalla Lg|500000|0
ABC-005|Mouse Logitech |180000|100

Por ejemplo, 999999 centavos son $9999.99 MXN, el precio máximo permitido, y 1 centavo es $0.01 MXN, el mínimo.

Para guardar, el programa arma todo el texto en un buffer y usa CreateOutputFile, WriteToFile y CloseFile. Después compara los bytes escritos con los que se pidieron escribir para comprobar que se escribió todo.

Para cargar, usa OpenInputFile, ReadFromFile y CloseFile. Como ReadFromFile no agrega el 00h, se coloca en la posición que indica la cantidad de bytes leídos. Luego se recorre el texto con ESI y cada campo se valida con los mismos procedimientos que se usan al capturar desde el teclado: código con formato ABC-123 y sin repetirse, nombre de 1 a 25 caracteres, precio de 1 a 999999 centavos y existencia de 0 a 100. Todo se arma en el catálogo temporal y solo si el archivo completo es válido se copia al catálogo real. Si el archivo no existe, está vacío, es demasiado grande o tiene un error, se muestra el motivo y la línea, y el catálogo actual no cambia.

Decisiones investigadas

División: antes de cada DIV se pone EDX en 0, porque con un divisor de 32 bits el dividendo es EDX:EAX. El cociente queda en EAX y el residuo en EDX; si EDX trajera otro valor, el resultado sería incorrecto o el cociente no cabría en EAX.

Escritura del archivo: WriteToFile llama a la función WriteFile de Windows, que informa cuántos bytes escribió realmente (Microsoft Learn). Por eso se compara ese número con los bytes pedidos. La cantidad pedida se guarda en una variable porque WriteToFile no conserva ECX.

Validar el archivo en un catálogo temporal: así un archivo dañado no deja el catálogo a medias, como pide el proyecto.

Fuentes consultadas

- Medina Jiménez, J. L. Lenguajes de Interfaz, Unidad 2: Programación básica. Presentación del curso, agosto-diciembre 2026.
- Irvine, K. R. Assembly Language for x86 Processors, 8.ª ed. Pearson. Código fuente Irvine32.asm incluido con la biblioteca.
- DIV — Unsigned Divide. Extracto del Intel 64 and IA-32 Architectures Software Developer's Manual. https://www.felixcloutier.com/x86/div.html
- WriteFile function (fileapi.h). Microsoft Learn. https://learn.microsoft.com/windows/win32/api/fileapi/nf-fileapi-writefile